

Student 10 **GONTLA SAI DEEPAK** 192311432RD

5 / 5 submitted

Q1. Exception

Score: 10.00 TC: 3/3 passed

CODE

```
import java.util.Scanner;
public class Main{
    public static void main(String[] args){
        try{
            Scanner sc = new Scanner(System.in);

            int[] arr = new int[5];
            for(int i = 0; i < arr.length; i++){
                arr[i] = sc.nextInt();

            }

            int a = sc.nextInt();
            System.out.print(arr[a]);
        }
        catch(Exception e){
            System.out.println("Exception: Invalid array index.");
        }

    }
}
```

INPUT

10 20 30 40 50
6

OUTPUT

Exception: Invalid array index.

Q2. IllegalArgumentException

Score: 10.00 TC: 3/3 passed

CODE

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int marks = sc.nextInt();

        try {
            checkMarks(marks);
            System.out.println("Valid marks: " + marks);
        } catch (IllegalArgumentException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    static void checkMarks(int marks) {
        if (marks < 0 || marks > 100) {
            throw new IllegalArgumentException("Invalid marks. Marks should be between 0 and 100.");
        }
    }
}
```

INPUT

120

OUTPUT

Exception: Invalid marks. Marks should be between 0 and 100.

Q3. user-defined exception named WeakPasswordException.

Score: 10.00 TC: 3/3 passed

CODE

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String password = sc.nextLine();

        try {
            checkPassword(password);
            System.out.println("Password is valid.");
        } catch (WeakPasswordException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }

    static void checkPassword(String password) throws WeakPasswordException {
        if (password.length() < 8) {
            throw new WeakPasswordException("Password must contain at least 8 characters.");
        }
    }
}

class WeakPasswordException extends Exception {
    public WeakPasswordException(String message) {
        super(message);
    }
}
```

INPUT

java123

OUTPUT

Exception: Password must contain at least 8 characters.

Q4. Email Validation

Score: 10.00 TC: 3/3 passed

CODE

```
import java.util.Scanner;
public class Main {
    static void validateEmail(String email) throws InvalidEmailException {
        if (!email.contains("@")) {
            throw new InvalidEmailException("Invalid email address.");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String email = sc.nextLine();

        try {
            validateEmail(email);
            System.out.println("Valid email address.");
        } catch (InvalidEmailException e) {
            System.out.println("Exception: " + e.getMessage());
        }
    }
}

class InvalidEmailException extends Exception {
    public InvalidEmailException(String message){
        super(message);
    }
}
```

INPUT

student.gmail.com

OUTPUT

Exception: Invalid email address.

Q5. Banking Transaction Using Exception Propagation

Score: 0.00 TC: 0/3 passed

CODE

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {

            double balance = sc.nextDouble();

            double withdrawAmount = sc.nextDouble();

            BankingSystem bank = new BankingSystem(balance);
            try{
                bank.processTransaction(withdrawAmount);
            }
        } catch (InsufficientBalanceException e) {
            System.out.println("Exception propagated to main().");
            System.out.println("InsufficientBalanceException: " + e.getMessage());

        }
    }
}

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

class BankingSystem {
    double balance;

    BankingSystem(double balance) {
        this.balance = balance;
    }

    void processTransaction(double amount) throws InsufficientBalanceException {
        validateBalance(amount);
    }

    void validateBalance(double amount) throws InsufficientBalanceException {
        if (amount > balance) {
            throw new InsufficientBalanceException(
                "Withdrawal amount exceeds available balance."
            );
        }
        withdraw(amount);
    }

    void withdraw(double amount){
        balance = balance - amount;
        System.out.println("Transaction successful. Remaining balance: " + balance);
    }
}
```

INPUT5000
7000